Gestión de un parque eólico

Trabajo final · Primera entrega

> *—¿Qué gigantes? —dijo Sancho Panza.*
> *—Aquellos que allí ves —respondió su amo—, de los brazos largos, que los suelen tener algunos de casi dos leguas.*
>
> *—Mire vuestra merced —respondió Sancho— que aquellos que allí se parecen no son gigantes, sino molinos de viento, y lo que en ellos parecen brazos son las aspas, que, volteadas del viento, hacen andar la piedra del molino.*
>
> Miguel de Cervantes, Don Quijote de la Mancha. Primera parte, capítulo VIII.

El trabajo de la asignatura consiste en desarrollar una aplicación en Python para supervisar y gestionar un parque eólico. El proyecto se divide en tres entregas con una aproximación incremental. En esta primera parte se construirá la aplicación local, utilizando programación orientada a objetos, ficheros de texto y una interfaz de consola.

El parque estará organizado en áreas, cada una con varios aerogeneradores. El usuario podrá consultar su estado, activarlos, desactivarlos, cambiar su orientación y gestionar sus averías con un número limitado de equipos técnicos. La simulación avanzará por pasos y permitirá comprobar si cada área mantiene el mínimo de aerogeneradores activos requerido.

Descripción del funcionamiento general

Al iniciar el programa, se cargará la configuración desde un fichero de texto. Después aparecerá un menú desde el que el usuario gestionará el parque. La distribución de áreas y aerogeneradores permanecerá fija durante esa ejecución.

Las reglas de desgaste y mantenimiento son un modelo didáctico. No será necesario calcular producción eléctrica, costes ni fenómenos físicos. El cumplimiento del servicio se determinará únicamente a partir del número de aerogeneradores activos de cada área.

Todas las operaciones se realizarán localmente en esta entrega. Las comunicaciones de red y las estructuras de datos de las entregas posteriores se incorporarán más adelante.

Estados y operaciones del parque

Cada aerogenerador tendrá un identificador único, una orientación, un estado y un nivel de desgaste. Su umbral de avería permitirá representar equipos con distintas necesidades de mantenimiento. El área determinará cuánto desgaste acumula un equipo activo por paso.

Estado	Significado
ACTIVO	Está en funcionamiento y cuenta para el mínimo de servicio.
DESACTIVADO	Está parado, sin avería. Puede activarse u orientarse.
AVERIADO	Tiene una avería pendiente de atención y no puede activarse.
EN_REPARACION	Tiene un equipo técnico asignado hasta terminar la reparación.

Operaciones disponibles

- Consultar el estado de todo el parque, de un área o de un aerogenerador. Las consultas no modifican datos ni avanzan el tiempo.
- Activar un aerogenerador desactivado. Desactivar un aerogenerador activo. Las demás transiciones mediante estas operaciones se rechazarán.
- Orientar un aerogenerador desactivado hacia N, E, S u O. Solo se admiten estas cuatro cadenas exactas, escritas en mayúsculas. En esta entrega la orientación se registra, pero no modifica el desgaste ni el cumplimiento del servicio.
- Iniciar la reparación de un aerogenerador averiado si queda algún equipo técnico libre.
- Avanzar un paso de simulación y mostrar las nuevas averías, las reparaciones terminadas y la situación del servicio.
- Salir de la aplicación. En esta entrega no se exige guardar el estado al salir.

Una operación rechazada no modificará el estado del parque ni consumirá recursos. La interfaz explicará el motivo. Los identificadores inexistentes y las orientaciones no admitidas también deben rechazarse.

Comprobación del servicio

Un área cumple el servicio si su número de aerogeneradores ACTIVO es mayor o igual que su mínimo requerido. El parque cumple cuando todas sus áreas cumplen. Los equipos activos de un área no compensan la falta de equipos en otra.

Se permitirá desactivar un equipo aunque esto deje al área por debajo del mínimo. La aplicación mostrará la situación de servicio insuficiente. Este estado no termina la simulación: el usuario puede seguir actuando para recuperarlo.

Desgaste e incidencias

La simulación comienza en el paso 0. Solo la opción Avanzar al siguiente paso cambia el tiempo. Entre dos pasos se pueden realizar varias consultas y operaciones de gestión. No se utilizarán esperas reales, temporizadores ni generación aleatoria de averías.

Generación de averías

En cada paso, un aerogenerador que estaba ACTIVO al comenzar acumula tantos puntos de desgaste como indique la exigencia de su área. Cuando alcanza o supera su umbral, pasa a AVERIADO. El desgaste se limita al umbral y se genera un único aviso con el formato exacto «<identificador>: nueva avería <código>».

Para incluir dos tipos de incidencia de forma reproducible, cada equipo alternará entre E1 y E2: su primera avería será E1, la segunda E2, la tercera E1 y así sucesivamente. Este comportamiento se proporcionará implementado. La alternancia es una regla de la simulación, no un modelo físico.

Código	Incidencia simulada	Duración
E1	Fallo de sensor	1 paso
E2	Fallo del mecanismo de orientación	2 pasos

Los equipos DESACTIVADO, AVERIADO o EN_REPARACION no acumulan desgaste. Un equipo averiado conserva el mismo código hasta que termina su reparación; las consultas posteriores no generan otra incidencia.

Equipos técnicos y reparación

Cada reparación ocupa un equipo técnico. Si todos están ocupados, la solicitud se rechaza y el aerogenerador permanece AVERIADO. El usuario podrá solicitarlo de nuevo más adelante. En esta entrega no habrá asignación automática ni cola de espera.

Al iniciar una reparación, el estado pasa a EN_REPARACION y se fija su duración según el código. Cada avance reduce en uno los pasos pendientes. Al llegar a cero, el equipo queda DESACTIVADO, su

desgaste vuelve a cero, su código se vacía y el técnico queda libre. La orientación se conserva.
El usuario deberá activarlo si quiere ponerlo en marcha.

Orden de un paso

- Incrementar en uno el contador de pasos.
- Actualizar una vez cada aerogenerador. Las áreas se recorren en el orden de sus registros AREA y, dentro de cada una, los aerogeneradores en el orden de sus registros AEROGENERADOR. No se ordenarán alfabéticamente ni por identificador. Se aplicará únicamente la regla del estado que tenía cada equipo al comenzar el paso.
- Recalcular los técnicos disponibles y el cumplimiento del servicio. Mostrar los avisos generados.

Un equipo que termina su reparación durante un paso no acumula desgaste en ese mismo paso. Una avería que aparece al avanzar solo podrá recibir una orden de reparación después de completar ese avance.

Carga de la configuración

La aplicación recibirá la ruta del fichero como argumento al ejecutarse. Se utilizará texto UTF-8, una línea por registro y el carácter ; como separador. Los registros aparecerán en este orden: equipos técnicos, áreas y aerogeneradores. No habrá cabecera, comentarios ni líneas vacías. Los registros AREA determinarán el orden de las áreas. Cada aerogenerador se añadirá a su área al procesar su registro y conservará ese orden de carga.

```
python3 app.py escenarios/parque_inicial.txt
```

Todos los ficheros proporcionados existirán y serán correctos en formato y contenido: valores admitidos, identificadores únicos y referencias a áreas existentes. No se exige validar esos ficheros ni gestionar errores de apertura. Esta garantía no elimina las validaciones de las operaciones del usuario.

Formato de los registros

```
TECNICOS;cantidad
AREA;nombre;exigencia;minimo_activos
AEROGENERADOR;id;area;estado;orientacion;desgaste;umbral
```

Cantidad, exigencia, mínimo y umbral serán enteros positivos. El mínimo no superará el número de equipos del área. El desgaste inicial será un entero mayor o igual que cero y menor que el umbral.

Los estados iniciales serán ACTIVO o DESACTIVADO; las orientaciones serán N, E, S u O. Los nombres no contendrán el separador.

El contador de pasos y el número de averías de cada equipo comenzarán en cero. Todos los técnicos estarán libres, los códigos de avería estarán vacíos y no habrá reparaciones iniciadas.

Ejemplo de configuración

```
TECNICOS;1
AREA;Norte;2;2
AREA;Sur;1;1
AEROGENERADOR;N1;Norte;ACTIVO;N;4;6
AEROGENERADOR;N2;Norte;ACTIVO;N;0;10
AEROGENERADOR;N3;Norte;ACTIVO;N;0;12
AEROGENERADOR;S1;Sur;ACTIVO;S;2;4
AEROGENERADOR;S2;Sur;DESACTIVADO;S;0;8
```

La carga debe crear las instancias de las clases y establecer sus relaciones. No se permite programar un caso particular para cada escenario, ni fijar en el código los nombres, cantidades o parámetros del fichero.

Se proporcionarán varios escenarios para explorar el comportamiento. Las comprobaciones funcionales podrán combinar un fichero válido con una secuencia concreta de acciones. Alcanzar temporalmente un servicio insuficiente no implica que el programa sea incorrecto: debe reflejarlo fielmente y permitir gestionar la situación.

Estructura del código

El módulo parque.py contendrá las clases Aerogenerador, Area y ParqueEolico. app.py implementará la interacción por consola. Los métodos de las clases no pedirán datos con input(). Podrán añadirse atributos y métodos auxiliares, respetando las firmas obligatorias.

Las firmas siguientes muestran los parámetros además de self. Se utilizarán las anotaciones de tipos indicadas. Los constructores almacenarán los argumentos y crearán las colecciones propias de cada instancia.

Aerogenerador

```
__init__(identificador: str, estado: str, orientacion: str, desgaste: int, umbral: int)
consultar_estado() -> str
activar() -> bool
desactivar() -> bool
orientar(orientacion: str) -> bool
iniciar_reparacion() -> bool
```

avanzar_paso(exigencia: int) -> str

Atributos obligatorios: identificador: str, estado: str, orientacion: str, desgaste: int, umbral: int, codigo_averia: str, averias_acumuladas: int y pasos_reparacion: int. Los tres últimos se inicializan a "", 0 y 0.

Se proporcionará avanzar_paso implementado: acumula desgaste, genera averías o progresa una reparación según el estado inicial. Devolverá exactamente «<identificador>: nueva avería <código>» cuando aparezca una avería, «<identificador>: reparación terminada» cuando finalice una reparación y la cadena vacía "" si no ocurre ninguno de estos sucesos. El estudiante completará el constructor y el resto de métodos.

consultar_estado() devuelve exclusivamente una de estas cadenas: ACTIVO, DESACTIVADO, AVERIADO o EN_REPARACION. Activar, desactivar, orientar e iniciar_reparacion devuelven True si realizan la operación y False si no está permitida, sin modificar ningún dato. orientar() solo acepta N, E, S u O, en mayúsculas; cualquier otro valor devuelve False. Orientar hacia la orientación actual, estando desactivado, es válido. iniciar_reparacion() prepara el contador según E1 o E2; la disponibilidad de técnicos la comprueba ParqueEolico antes de llamar a este método.

Area

__init__(nombre: str, exigencia: int, minimo_activos: int)
anadir_aerogenerador(aerogenerador: Aerogenerador) -> None
contar_activos() -> int
cumple_servicio() -> bool

Atributos obligatorios: nombre: str, exigencia: int, minimo_activos: int y aerogeneradores: list[Aerogenerador]. La lista comienza vacía. Añadir conserva el orden de carga. Contar devuelve únicamente el número de equipos ACTIVO; cumple_servicio compara ese número con el mínimo.

Control del parque e interfaz

ParqueEolico

__init__()
cargar_configuracion(ruta: str) -> None
buscar_aerogenerador(identificador: str) -> Aerogenerador | None
activar_aerogenerador(identificador: str) -> bool
desactivar_aerogenerador(identificador: str) -> bool
orientar_aerogenerador(identificador: str, orientacion: str) -> bool
reparar_aerogenerador(identificador: str) -> bool
tecnicos_disponibles() -> int
cumple_servicio() -> bool

avanzar_paso() -> list[str]

Atributos obligatorios: areas: list[Area], total_tecnicos: int y paso_actual: int. El constructor crea una lista vacía y fija ambos enteros a cero. La carga establece el parque desde el fichero y reinicia la simulación; si ya había datos, los sustituye.

Buscar devuelve el objeto con ese identificador o None. Las operaciones por identificador devuelven False si el equipo no existe o la operación no está permitida. La reparación requiere además un técnico libre. Las demás operaciones delegan en el aerogenerador encontrado.

Los técnicos disponibles se calculan restando al total el número de equipos EN_REPARACION. Cumple_servicio devuelve True si todas las áreas cumplen; antes de cargar un parque, devuelve False. Avanzar_paso actualiza todos los equipos y devuelve los avisos no vacíos en el orden de recorrido. Si no hay avisos, devuelve una lista vacía.

Punto de entrada y menú

Se proporcionará un app.py base con un main() que no se podrá modificar. Este cargará el fichero indicado y llamará a las funciones de menú, que completará el estudiante. La aplicación mostrará el paso actual, los técnicos disponibles y el cumplimiento del servicio.

```
=== Parque eólico ===
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
```

Las consultas respetarán el orden de carga y mostrarán como mínimo la información siguiente:

- Consultar parque: todas las áreas y todos sus aerogeneradores. Para cada área se mostrarán el nombre, la exigencia, el número de activos, el mínimo requerido y si cumple el servicio.
- Consultar área: los datos anteriores del área solicitada y todos sus aerogeneradores.
- Consultar aerogenerador: identificador, estado, orientación y desgaste actual respecto del umbral. Si está AVERIADO o EN_REPARACION se mostrará también el código; si está EN_REPARACION, además se indicarán los pasos pendientes.

No se exige un formato textual exacto para estas consultas: podrán cambiar los encabezados, separadores y espacios, siempre que aparezcan los datos requeridos. Las opciones desconocidas, los identificadores inexistentes y las operaciones rechazadas mostrarán un mensaje y permitirán continuar.

Ejemplos de consultas

Las siguientes capturas ilustran una posible interfaz. Se muestran el contexto del menú, la selección numerada y los datos mínimos de cada consulta. El formato visual puede variar siempre que se respete la información exigida.



Figura 1. Consulta del estado inicial del parque cargado desde el fichero de configuración.

```
=== Parque eólico ===
Paso: 0
Técnicos disponibles: 1
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
> 2

Áreas disponibles:
1) Norte
2) Sur
Seleccione un área: 2

Área Sur
Exigencia: 1
Activos: 1/1 requeridos
Servicio: CUMPLE
  S1 | ACTIVO | orientación S | desgaste 2/4
  S2 | DESACTIVADO | orientación S | desgaste 0/8

=== Parque eólico ===
Paso: 0
Técnicos disponibles: 1
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
>
```

Figura 2. Selección numerada y consulta del área Sur.

```
=== Parque eólico ===
Paso: 0
Técnicos disponibles: 1
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
> 3

Aerogeneradores disponibles:
1) N1 – Área Norte
2) N2 – Área Norte
3) N3 – Área Norte
4) S1 – Área Sur
5) S2 – Área Sur
Seleccione un aerogenerador: 1
N1 | ACTIVO | orientación N | desgaste 4/6

=== Parque eólico ===
Paso: 0
Técnicos disponibles: 1
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
>
```

Figura 3. Selección numerada y consulta del aerogenerador N1.

Comprobaciones y Entrega

Material de apoyo previsto

- El enunciado y los ficheros de configuración de los escenarios.
- La clase Aerogenerador parcialmente implementada y el app.py base.
- Un test público de estructura para comprobar las clases y sus interfaces.

El test de estructura comprobará la existencia de clases, constructores, métodos, parámetros y tipos. Podrá crear objetos y realizar llamadas básicas para verificar atributos y valores de retorno. Las colecciones deben pertenecer a cada instancia y los métodos no deben solicitar datos por teclado.

Superar el test de estructura no garantiza el funcionamiento completo. También se comprobarán la carga, las transiciones, la evolución del desgaste, la duración de las reparaciones, la disponibilidad de técnicos y el servicio por área. Los tests funcionales podrán comparar de forma exacta estados, atributos, valores booleanos, listas y los avisos devueltos por avanzar_paso(). En la interfaz de consola se comprobará que aparecen los datos requeridos, pero no una cadena completa exacta ni

detalles de formato como espacios, separadores o encabezados. Los escenarios de prueba respetarán las reglas de este enunciado.

Entrega del trabajo

El proyecto se entregará a través de Aula Virtual en un único archivo ZIP que incluya los archivos .py y los escenarios .txt necesarios para ejecutarlo. No se incluirán entornos virtuales ni carpetas de caché.

Fecha límite: pendiente de publicación en Aula Virtual. La composición de los equipos y las condiciones de evaluación serán las indicadas para este curso en Aula Virtual y en la presentación de la asignatura.

Anexo de ejecución de ejemplo

Con el fichero de ejemplo, el parque comienza cumpliendo: Norte tiene 3 activos y necesita 2; Sur tiene 1 y necesita 1. Hay un técnico libre. La siguiente secuencia ilustra el orden de actualización.

Acción	Resultado esperado
Avanzar al paso 1	N1 alcanza 6 de desgaste: avería E1. N2=2, N3=2 y S1=3. Ambas áreas cumplen.
Reparar N1	N1 queda EN_REPARACION con 1 paso pendiente. Quedan 0 técnicos libres.
Avanzar al paso 2	N1 termina: DESACTIVADO, desgaste 0. S1 alcanza 4: E1. N2=4 y N3=4. Sur incumple. Hay 1 técnico libre.
Activar S2 y reparar S1	Sur recupera su mínimo. S1 queda EN_REPARACION con 1 paso pendiente.
Avanzar al paso 3	S1 termina desactivado y sin desgaste. S2 acumula 1. N2=6 y N3=6. El parque cumple.

Para comprobar E2, un aerogenerador reparado y reactivado deberá alcanzar de nuevo su umbral. Esa segunda avería requerirá dos avances desde que se inicie la reparación.

Secuencia de avance y reparación

Las capturas siguientes corresponden a la secuencia resumida en la tabla. Iniciar una reparación asigna un técnico y cambia el estado del aerogenerador, pero la reparación solo progresa cuando se selecciona Avanzar al siguiente paso.

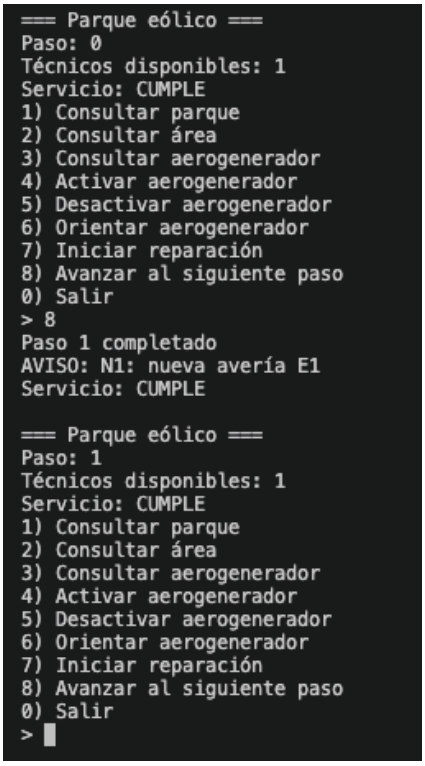


Figura 4. El paso 1 genera la avería E1 de N1.

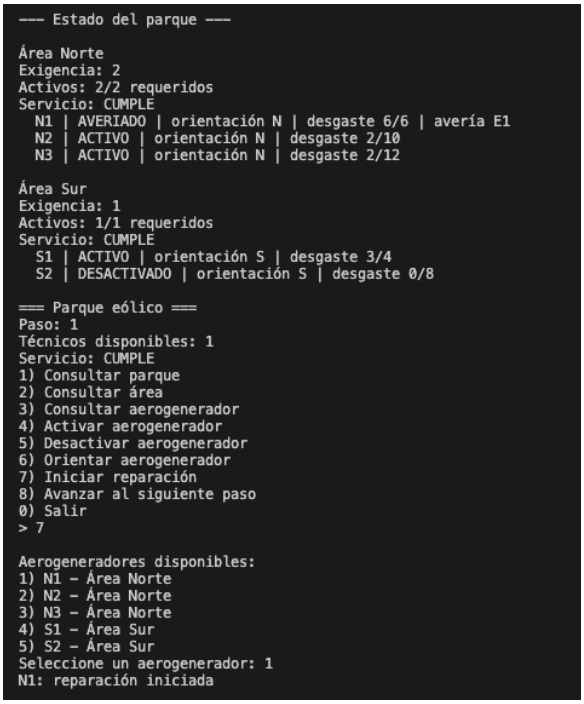


Figura 5. Consulta del parque tras el avance: N1 aparece AVERIADO.

```
--- Estado del parque ---

Área Norte
Exigencia: 2
Activos: 2/2 requeridos
Servicio: CUMPLE
  N1 | AVERIADO | orientación N | desgaste 6/6 | avería E1
  N2 | ACTIVO | orientación N | desgaste 2/10
  N3 | ACTIVO | orientación N | desgaste 2/12

Área Sur
Exigencia: 1
Activos: 1/1 requeridos
Servicio: CUMPLE
  S1 | ACTIVO | orientación S | desgaste 3/4
  S2 | DESACTIVADO | orientación S | desgaste 0/8

=== Parque eólico ===
Paso: 1
Técnicos disponibles: 1
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
> 7

Aerogeneradores disponibles:
1) N1 – Área Norte
2) N2 – Área Norte
3) N3 – Área Norte
4) S1 – Área Sur
5) S2 – Área Sur
Seleccione un aerogenerador: 1
N1: reparación iniciada
```

Figura 6. La aplicación confirma el inicio de la reparación de N1.

```
--- Estado del parque ---

Área Norte
Exigencia: 2
Activos: 2/2 requeridos
Servicio: CUMPLE
  N1 | EN_REPARACION | orientación N | desgaste 6/6 | avería E1 | pasos pendientes 1
  N2 | ACTIVO | orientación N | desgaste 2/10
  N3 | ACTIVO | orientación N | desgaste 2/12

Área Sur
Exigencia: 1
Activos: 1/1 requeridos
Servicio: CUMPLE
  S1 | ACTIVO | orientación S | desgaste 3/4
  S2 | DESACTIVADO | orientación S | desgaste 0/8

=== Parque eólico ===
Paso: 1
Técnicos disponibles: 0
Servicio: CUMPLE
1) Consultar parque
2) Consultar área
3) Consultar aerogenerador
4) Activar aerogenerador
5) Desactivar aerogenerador
6) Orientar aerogenerador
7) Iniciar reparación
8) Avanzar al siguiente paso
0) Salir
> 8
Paso 2 completado
AVISO: N1: reparación terminada
AVISO: S1: nueva avería E1
Servicio: INSUFICIENTE
```

Figura 7. Al avanzar al paso 2 termina la reparación de N1 y aparece una nueva avería en S1.